EC 作成者：eirin cho

Made with GAMMA

Agent Skillsとは？

Anthropic社は、Skillについて次のように定義しています。

> “Skills are organized folders of instructions, scripts, and resources that agents can discover and load dynamically.”
>
> 「スキルとは、エージェントが動的に発見しロードできる、指示、スクリプト、リソースが整理されたフォルダのことです。」
>
> — Anthropic

🧠 最も簡単な理解

概念	イメージ
Agent	🧠 脳
MCP	🖐️ 手＋ツール
Skill	📖 作業方法・SOP・専門知識

例えば：

Agent：

「プロフェッショナルなPPTを作りたい。」

MCP：

「ファイルを操作したり、ツールを呼び出したり、データを取得したりできます。」

Skill：

「PPTを作成する場合は、まず対象者を分析する → 構成を決める → 各スライドの内容を設計する → 文字量を調整する → グラフを追加する → 最後にチェックする。」

つまり、次のようにまとめることができます。

MCPは、Agentに「何ができるか」を与える。

Skillは、Agentに「どのようにやるべきか」を教える。

Made with GAMMA

Agent Skillsの進化：プロンプトによる指示の限界

Agent Skillsの概念が生まれる前、AIにタスクを指示する最も基本的な方法は、プロンプトに直接すべてを書き込むことでした。しかし、このアプローチにはいくつかの課題がありました。

第一段階：プロンプトでの方法記述

初期の段階では、AIに対して「以下の手順に従ってプレゼンテーション資料を作成してください...」のように、タスクの実行方法をプロンプト内に詳細に記述していました。

> *「以下の方法でプレゼンテーションを作成してください：まず聴衆を分析し、構成を決定し、各ページをデザインし、文字量を調整し、図表を追加し、最終チェックを行ってください。」*

このアプローチはシンプルに見えますが、すぐに問題に直面しました。

- プロンプトが肥大化し、非常に長くなる傾向がありました。
- 同じ手順を必要とするタスクのたびに、プロンプトの内容を繰り返して記述する必要がありました。
- 一度作成した指示は再利用が困難で、効率が低下しました。
- 異なる種類のタスクに必要な知識や手順が、一つのプロンプトの中に混在しやすくなりました。

このように、知識や作業方法がすべてプロンプトに直接埋め込まれてしまうことが、大きな課題となっていたのです。

Made with GAMMA

Agent Skillsの進化

第二段階：ツールの導入

Agent Skillsの発展において、次の段階はAgentに「ツール」が与えられたことです。すなわち、MCP（Model Context Protocol）が与えられたことで、Agentは多様な操作を実行する「能力」を獲得しました。

1

Agentに能力が付与

Agentがファイル操作、GitHub連携、データベースアクセス、API呼び出しといった外部ツールを使用できるようになりました。

2

「操作可否」の問題解決

これにより「Agentは、この操作を実行できるのか？」という根本的な疑問が解消されました。

しかし、新たな課題が浮上しました。

> 「ツールがある」ことと「ツールを専門的に使いこなす方法を知っている」ことは異なります。

これは、「誰かにExcelを与えたとしても、その人が財務分析ができるわけではない」のと同じです。Agentも同様に、ツールがあっても、それをプロフェッショナルなレベルで活用する知識がなければ、その真価を発揮できません。

Made with GAMMA

Agent Skillsの進化

第三段階：複雑なタスクへの挑戦

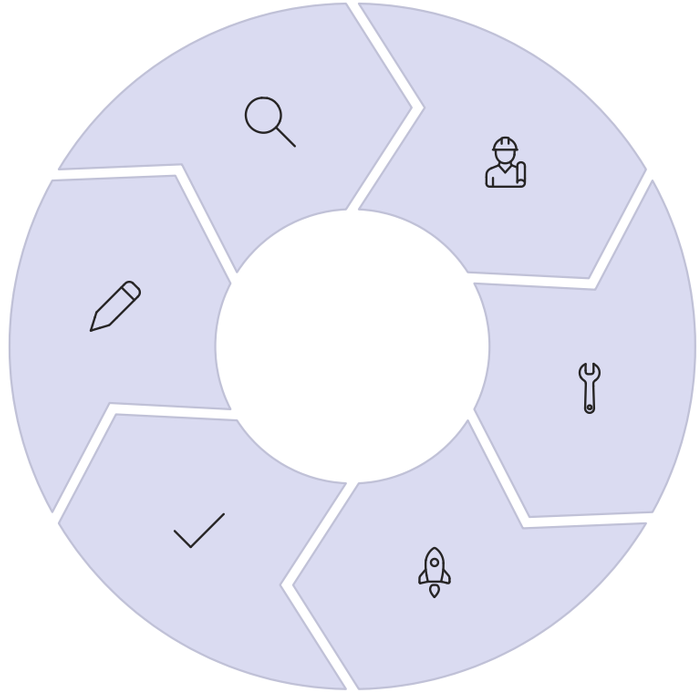
Agentにツールが与えられたことで、その役割は単純な「プロンプト→回答」の枠を超え、自律的に複雑なタスクを遂行する段階へと進化しました。

タスクの理解

与えられたタスクの要件を深く分析し、何をすべきかを把握します。

計画の策定

タスク達成のための具体的な手順や戦略を考案し、計画を立てます。



修正

必要に応じて計画や実行内容を見直し、改善を施します。

ツール呼び出し

必要な外部ツールやAPIを適切に選択し、それらを活用します。

確認

実行結果が期待通りであるかを評価し、問題がないかを確認します。

実行

策定した計画に基づき、各ステップを順次実行していきます。

しかし、タスクが複雑化するにつれて、Agentが自律的にこれらの工程を進めるためには、より高度な専門知識と洗練された操作プロセスが求められるようになりました。

> 例えば、単に「プレゼンテーションを作成する」のと、「会社のブランドガイドラインに沿って、20ページの技術紹介プレゼンテーションを作成する」のとでは、その難易度は大きく異なります。

Made with GAMMA

Agent Skills

第四段階：専門能力のモジュール化

Agent Skillsの発展は、専門知識の扱い方におけるパラダイムシフトをもたらしました。2025年、Anthropicは「Agent Skills」を発表し、その核心は、プロンプトにすべての専門知識を直接記述するのではなく、**モジュール化されたスキル**として整理することにありました。

このアプローチにより、Agentは必要に応じて特定のタスクに特化したスキルを動的に発見し、ロードできるようになります。これにより、より効率的かつ専門的なタスク遂行が可能となります。

スキルは、以下のような構造で構成されます。

```
skills/
├─── ppt/
│    ├─── SKILL.md
│    ├─── templates/
│    ├─── examples/
│    └─── scripts/
│
├─── excel/
│    ├─── SKILL.md
│    ├─── templates/
│    └─── scripts/
│
└─── pdf/
     ├─── SKILL.md
     └─── scripts/
```

Agentがタスクを受け取る：

「PPTを作成してください。」

↓

Agentは、利用できるSkillの中から、

PPT Skill

を見つけることができます。

↓

そして、そのSkillに含まれている
instructions、scripts、resources を読み込みます。

⭐ これにより、PPT作成のプロセスを、より明確で論理的な形にすることができます。

Prompt
↓
Tools
↓
MCP
↓
Agent
↓
Skills

ただし、ここで重要なのは、

「前の仕組みが、後の仕組みによって不要になる」ということではありません。

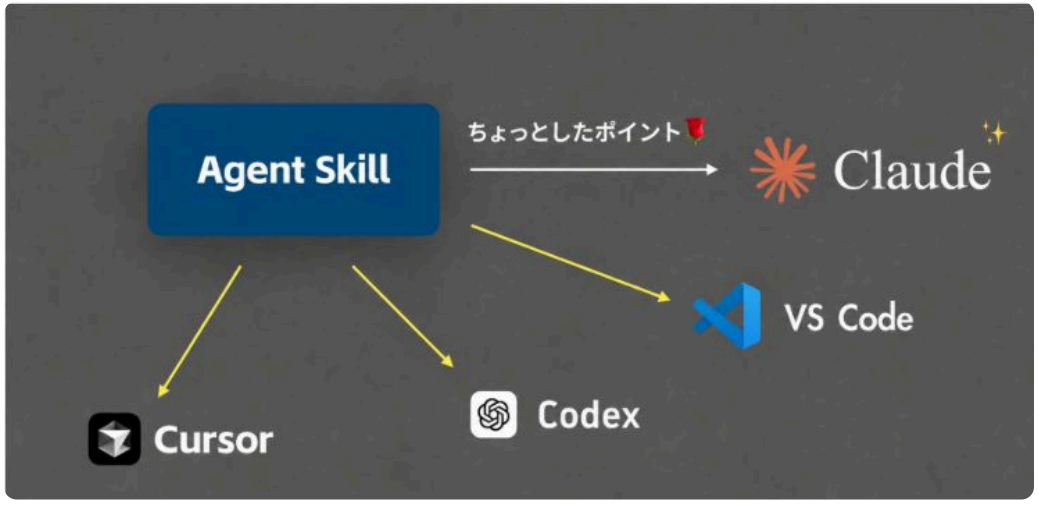
それぞれが役割分担をするようになります。

Made with GAMMA

Agent Skillsの進化

第五段階：ツール連携と普及

Anthropic社は、2025年12月18日にAgent Skillsをオープンスタンダードとして正式に発表し、プラットフォームや製品をまたいだ相互利用を可能にしました。
Cursor、Codex、VS Code、Claude CodeはいずれもAgent Skillsに対応しています。



Made with GAMMA

Agent Skillsの活用：基本的な使い方

Agent Skillsの力を最大限に引き出すためには、Claude Codeを使ってカスタムスキルを効率的に作成・管理することが重要です。ここでは、「会議まとめアシスタント」というスキルを例に、その基本的なセットアップ方法をステップバイステップでご紹介します。

まず、Claude Code内でスキルを整理するためのフォルダ構造を作成します。

```
cd "C:\Users\zhao yinglin\.claude"
mkdir skills
mkdir "会議まとめアシスタント"
```

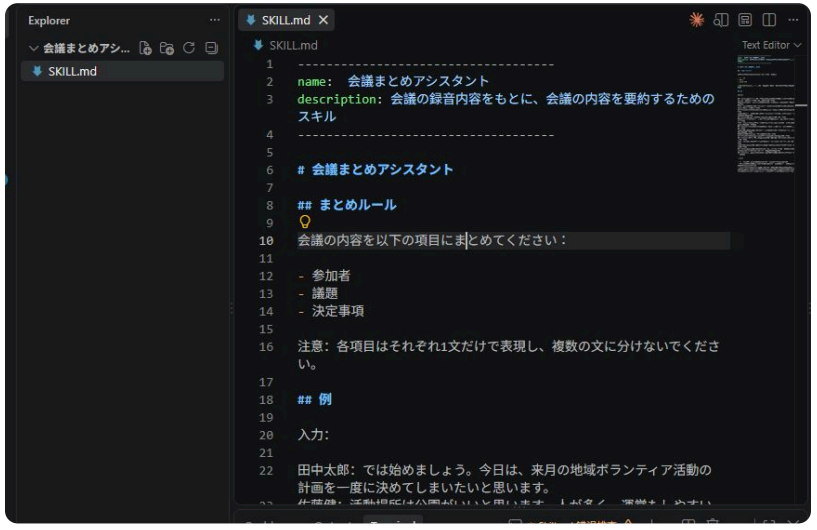
これらのコマンドを実行することで、Agentが動的に発見しロードできる新しいスキルを整理するための基盤が確立されます。この「会議まとめアシスタント」フォルダの中に、スキルの指示やスクリプト、リソースなどを配置していくことになります。

Made with GAMMA

Agent Skillsの活用：SKILL.mdの作成

前段階で作成したスキルのフォルダの中に、そのスキルの「定義」を記述するファイル、SKILL.mdを作成します。このファイルは、Agentがスキルを理解し、適切に利用するための中心的な役割を果たします。

SKILL.mdには、スキルの目的、使用方法、期待される入力と出力、そして実行手順などが詳細に記述されます。これは、スキルの「取扱説明書」のようなものです。



このSKILL.mdが、Agentに特定のタスクを「どのように」実行すべきかを教えるための**ガイド**となります。

Made with GAMMA

「SKILL.md」の定義：スキルの設計図

各Agent Skillには、このようなMDファイルを用意する必要があります。このファイルには、Agent Skillの名前、何ができるのか、そしてどのようにそのタスクを実行するのかを記述します。

例えば、今回作成したSkillは、**会議の録音内容を要約するためのもの**です。

---
name: 会議まとめアシスタント
description: 会議の録音内容をもとに、会議の内容を要約するためのスキル

会議まとめアシスタント

まとめルール

会議の内容を以下の項目にまとめてください：

- 参加者
- 議題
- 決定事項

注意：各項目はそれぞれ1文だけで表現し、複数の文に分けないでください。

.....

Made with GAMMA

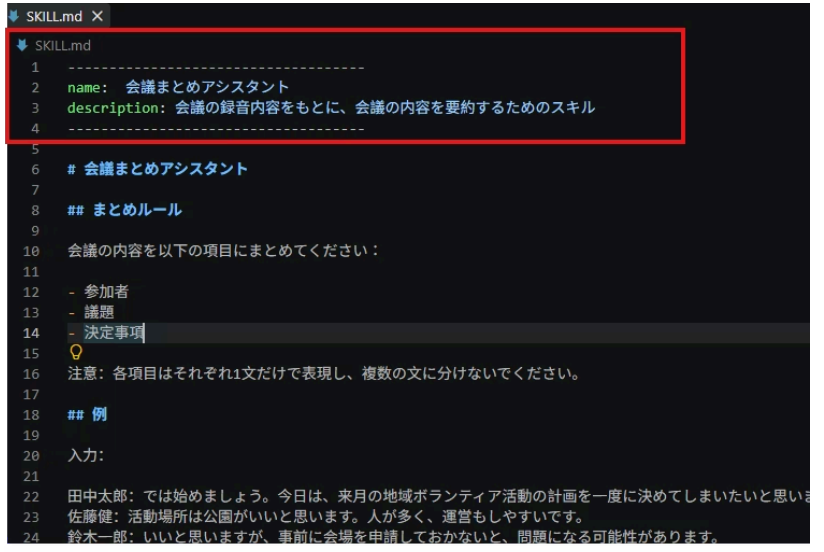
「SKILL.md」の定義：メタデータ

SKILL.mdファイルは、主に2つの部分から構成されます。まず、ファイルの冒頭にはそのスキルに関する基本的な情報が記述されており、これを「メタデータ」と呼びます。

このメタデータ部分には、主に以下の2つの属性が含まれます。

- **name:** Agent Skillの名称です。基本的に、Skillを格納するフォルダ名と同じ名称を使用します。
- **description:** このAgent Skillがどのような目的で、何を実行するのかを大規模言語モデル（LLM）に説明するための記述です。

これにより、Agentはスキルの概要と目的を素早く理解し、適切に利用する判断を下すことができます。



Made with GAMMA

「SKILL.md」の定義：実行ルールと具体的な指示

SKILL.mdファイルには、メタデータに加えて、Agentがタスクをどのように実行すべきかを具体的に定義する「実行ルールと具体的な指示」のセクションが含まれています。

この部分は、Agentが従うべきコアロジックや手順を記述するもので、大規模言語モデル（LLM）がタスクを遂行する際の「取扱説明書」となります。

公式ウェブサイトではこの部分を「指令」(instruction)と呼んでおり、モデルが遵守すべきルールを詳細に記述します。例えば、「会議まとめアシスタント」のスキルでは、以下のような具体的なまとめルールが設定されています。

```
# 会議まとめアシスタント

## まとめルール

会議の内容を以下の項目にまとめてください：

- 参加者
- 議題
- 決定事項

注意：各項目はそれぞれ1文だけで表現し、複数の文に分けないでください。
```

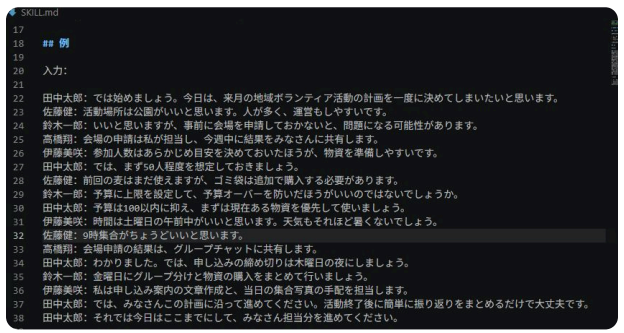
このように、SKILL.mdのこのセクションが、Agentに特定のタスクを「どのように」実行すべきかを指示する中核となります。

Made with GAMMA

「SKILL.md」の定義：入力データの例

大規模言語モデル（LLM）がタスクを適切に理解し実行するためには、定義されたルールだけでなく、実際の入力データを示すことも重要です。

「会議まとめアシスタント」のスキルにおいては、以下に示すような具体的な会議の内容がAgentに提供されます。



Made with GAMMA

「SKILL.md」の定義：出力結果の例

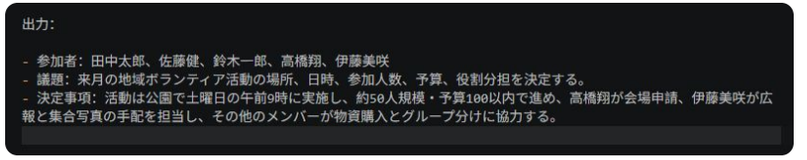
前述のSKILL.mdで定義されたルールと、提供された入力データに基づき、Agentは以下のような要約を生成します。

出力：

- 参加者：田中太郎、佐藤健、鈴木一郎、高橋翔、伊藤美咲
- 議題：来月の地域ボランティア活動の場所、日時、参加人数、予算、役割分担を決定する。
- 決定事項：活動は公園で土曜日の午前9時に実施し、約50人規模・予算100以内で進め、高橋翔が会場申請、伊藤美咲が広報と集合写真の手配を担当し、その他のメンバーが物資購入とグループ分けに協力する。

この出力は、SKILL.mdに記述された「各項目はそれぞれ1文だけで表現し、複数の文に分けないでください」という指示を厳密に守り、簡潔かつ的確に会議の内容を伝えています。

このように、ルールと具体的な入力例を組み合わせることで、Agentはタスクの意図をより深く把握し、期待される出力を高い精度で生成できるようになります。



このように、SKILL.mdはAgentがどのような形式でタスクを完了すべきかを明確にし、一貫性のある高品質な出力を実現します。

Made with GAMMA

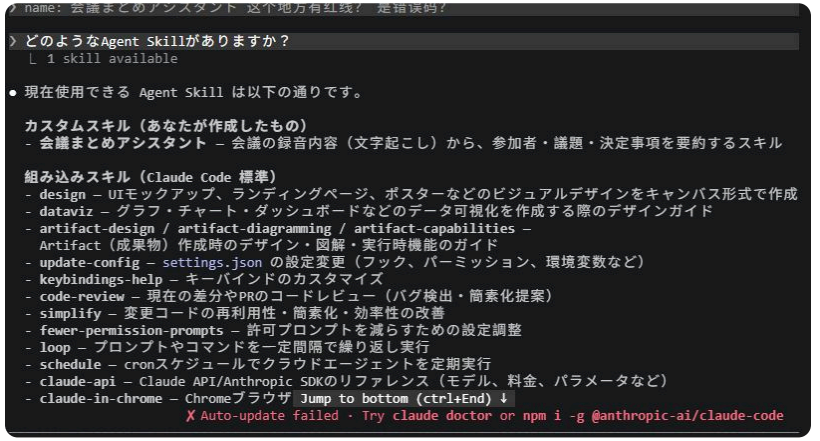
Agent Skillsの活用：簡単なテスト

「SKILL.md」ファイルが定義されたら、Claude Codeを使って簡単なテストを行いましょう。

まず、Claude Codeに「どのようなAgent Skillを持っていますか？」と質問します。これにより、Agentが正しくスキルを認識しているかを確認できます。

> Agent Skillを教えてください。

この質問に対し、Claude Codeは現在認識しているAgent Skillの一覧を返答します。ここに作成したスキルが含まれていることを確認しましょう。



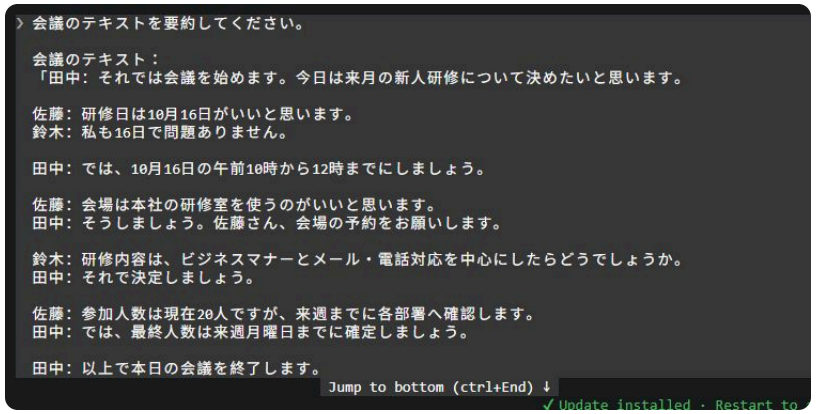
Made with GAMMA

Agent Skillsの活用：Claude Codeでの実行

SKILL.mdが正しく定義され、Agentに認識されたら、実際にClaude Codeを使ってスキルを実行してみましょう。会議の内容を要約するAgent Skillを使用するために、以下のようにClaude Codeにリクエストを送信します。

> 会議のテキストを要約してください。
>
> 会議のテキスト：
> 「田中：それでは会議を始めます。今日は来月の新人研修について決めたいと思います。
>
> 佐藤：研修日は10月16日がいいと思います。
> 鈴木：私も16日で問題ありません。
>
> 田中：では、10月16日の午前10時から12時までにしましょう。
>
> 佐藤：会場は本社の研修室を使うのがいいと思います。
> 田中：そうしましょう。佐藤さん、会場の予約をお願いします。
>
> 鈴木：研修内容は、ビジネスマナーとメール・電話対応を中心にしたらどうでしょうか。
> 田中：それで決定しましょう。
>
> 佐藤：参加人数は現在20人ですが、来週までに各部署へ確認します。
> 田中：では、最終人数は来週月曜日までに確定しましょう。
>
> 田中：以上で本日の会議を終了します。
> 」

このように、スキルを呼び出すプロンプトに続いて、要約したい会議のテキストをClaude Codeに貼り付けます。



これは、新入社員研修の日時、会場、研修内容、参加人数を決定するための会議です。

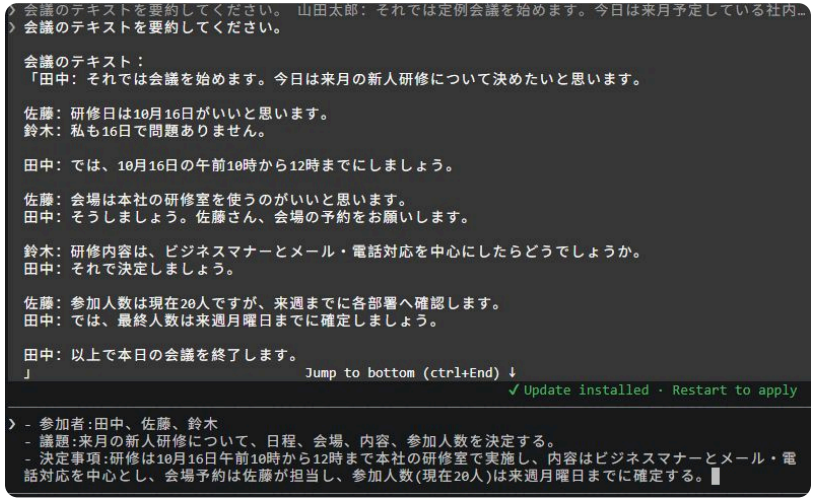
Made with GAMMA

Agent Skillsの活用：Claude Codeの出力

私たちが定義したSKILL.mdに厳密に従って、Claude Codeが生成した出力を見てみましょう。

> 出力：
> - 参加者:田中、佐藤、鈴木
> - 議題:来月の新人研修について、日程、会場、内容、参加人数を決定する。
> - 決定事項:研修は10月16日午前10時から12時まで本社の研修室で実施し、内容はビジネスマナーとメール・電話対応を中心とし、会場予約は佐藤が担当し、参加人数(現在20人)は来週月曜日までに確定する。

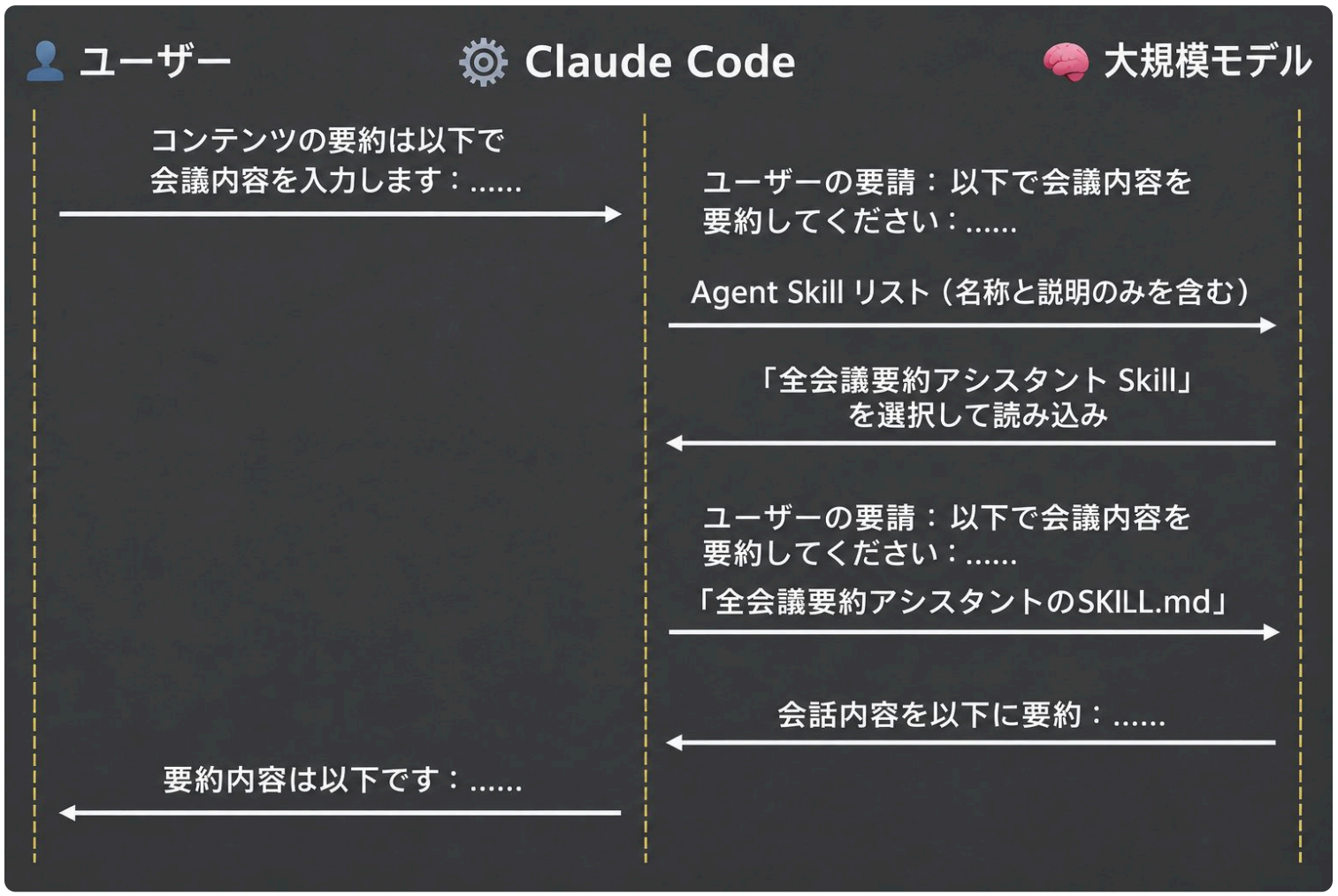
このように、SKILL.mdで設定した「参加者」「議題」「決定事項」の3つの項目が明確に分けられ、それぞれの内容が**1文で簡潔にまとめられています。**これは、期待通りの構造化された出力であり、Agentが指示を正確に理解し、実行したことを示しています。



この結果は、SKILL.mdがAgentに対してタスクの実行方法と出力形式をどれほど明確に指示できるかを示しています。これにより、複雑なテキストから必要な情報を効率的かつ一貫した形式で抽出することが可能になります。

Made with GAMMA

Claude CodeにおけるAgent Skillの利用フロー



全体の流れは、「ユーザー」「Claude Code」「モデル」の3つの役割で構成されています。

メリット：ここで、Agent Skillsの最初の重要な仕組みである「必要に応じた読み込み」につながります。

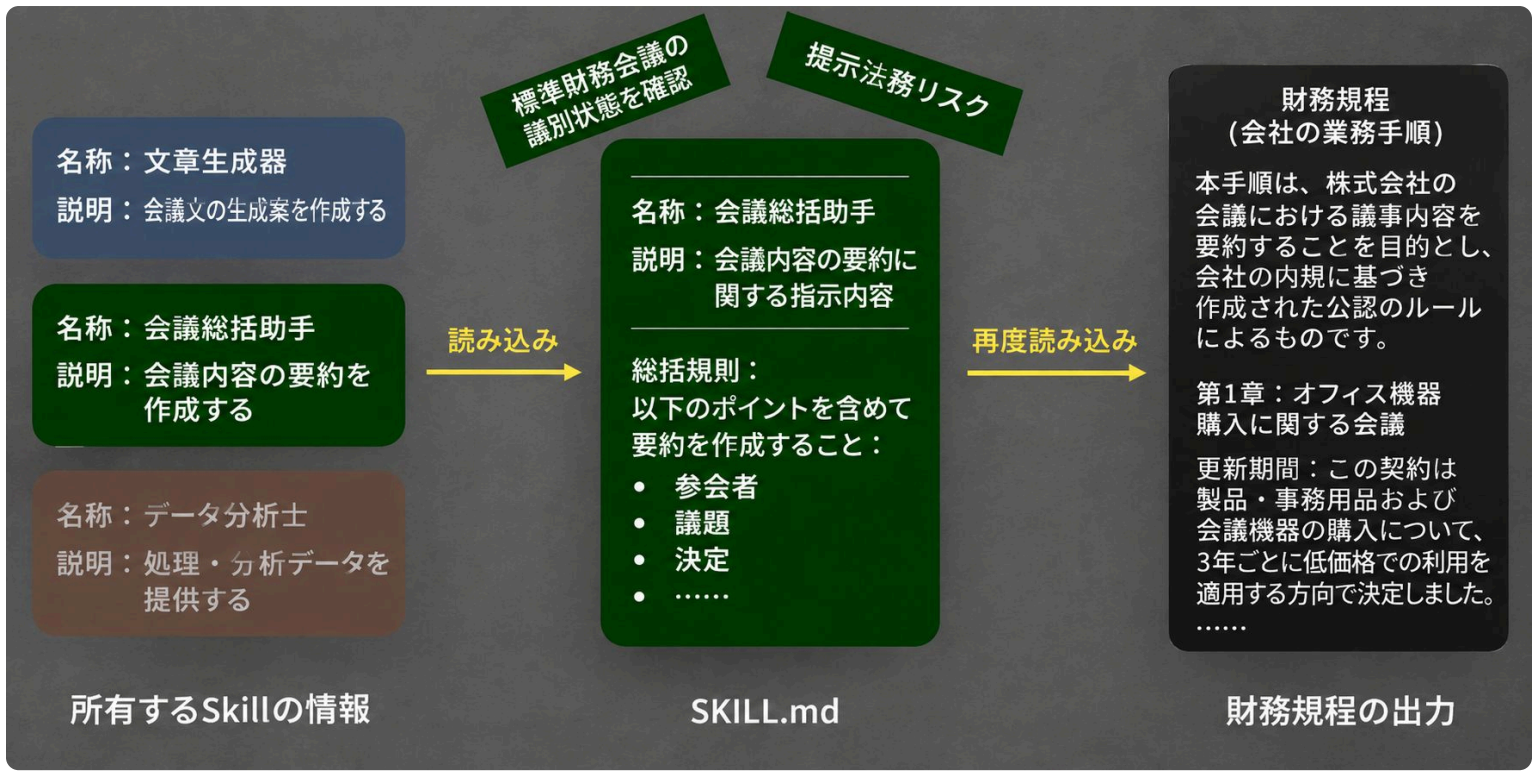
Skillの名前と説明は常にモデルから見える状態になっていますが、具体的な指示内容については、そのSkillが選択された後に初めて読み込まれ、モデルに渡されます。

これによって、必要のない情報を常にモデルに渡す必要がなくなるため、トークンを大幅に節約することができます。

Made with GAMMA

Skillの高度な使い方 — Reference編

Made with GAMMA



例えば、私たちの会議要約アシスタントが、今後もっと高度になっていったとします。

単純に会議の内容を要約するだけではなくて、会議の内容に応じて、さらに役立つ情報も追加してくれると便利ですよね。

例えば、会議の中で「お金を使う」という話が出てきたら、
「この決定は財務規定上、問題ないのか？」
ということまでチェックして、要約の中に表示してくれる。

また、「契約」という話が出てきたら、
「ここには法務上のリスクがあります」
と教えてくれる。

そうすれば、会議の要約を見るだけで、重要なポイントまで一緒に確認できます。

わざわざ別の資料を開いて、財務規定や法務ルールを確認する必要もなくなります。

かなり便利ですよね。

ただ、これをやろうとすると、一つ問題があります。

財務規定や法務ルールなどの情報を、全部SKILL.mdの中に書いておく必要があります。

そうすると、当然ですがSKILL.mdがどんどん長くなっていきます。

例えば、ただの朝会なのに、
「財務規定」や「法務ルール」まで全部モデルに読み込まれてしまう。

これでは、せっかくオンデマンドで読み込んでいるのに、まだ無駄がありますよね。

じゃあ、ここからさらに効率化できないのか。

例えば、会議の中で実際に「お金」の話が出たときだけ、財務に関する情報を追加で読み込む。

「契約」の話が出たときだけ、法務に関する情報を追加で読み込む。

こういうことができれば、もっと効率的ですよね。

実は、これができます。

Agent Skillsには「Reference」という仕組みがあります。

これを使うと、必要になったタイミングで、さらに必要な情報だけを読み込ませることができます。

それでは、実際に試してみましょう。

Claude Codeの場合、最初にAgent Skillsの名前と説明だけをモデルに渡して、その中から必要なSkillをモデル自身が選びます。

そして、選ばれたSkillのSKILL.mdだけを実際に読み込む仕組みになっています。

つまり、必要なものだけを読み込む「オンデマンドロード」ですね。

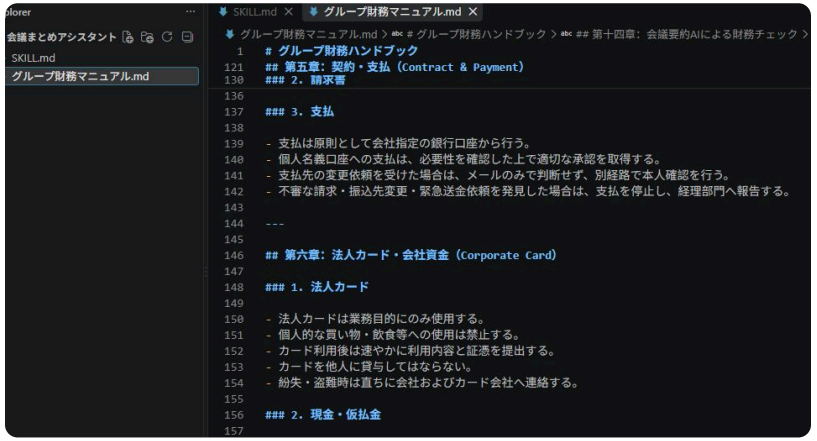
これだけでも、かなりトークンを節約できます。

ただ、ここでちょっと考えてみたいと思います。

Made with GAMMA

「Reference」の実装：財務マニュアルの追加

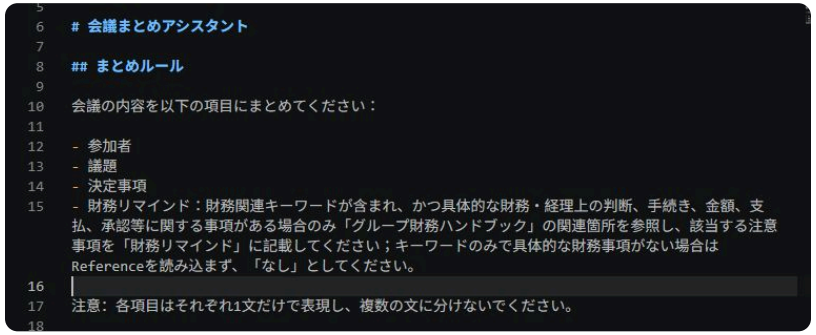
「Reference」機能の最初のステップとして、まず「グループ財務マニュアル」というファイルを追加します。このマニュアルには、様々な費用の精算基準や財務規定が詳細に記述されており、Agentが会議の内容に応じて必要な情報を参照できるようにします。



Made with GAMMA

「SKILL.md」に財務リマインド規則を追加

「Reference」機能の活用として、今度はSKILL.mdに財務リマインド規則を追加します。これにより、Agentは会議の内容に財務関連のキーワードが含まれ、かつ具体的な判断事項がある場合にのみ、「グループ財務ハンドブック」を参照し、関連する注意事項を抽出できるようになります。



```
# 会議まとめアシスタント

## まとめルール

会議の内容を以下の項目にまとめてください：

- 参加者
- 議題
- 決定事項
- 財務リマインド：財務関連キーワードが含まれ、かつ具体的な財務・経理上の判断、手続き、金額、支払、承認等に関する事項がある場合のみ「グループ財務ハンドブック」の関連箇所を参照し、該当する注意事項を「財務リマインド」に記載してください；キーワードのみで具体的な財務事項がない場合はReferenceを読み込まず、「なし」としてください。

注意：各項目はそれぞれ1文だけで表現し、複数の文に分けないでください。
```

> 財務リマインド：財務関連キーワードが含まれ、かつ具体的な財務・経理上の判断、手続き、金額、支払、承認等に関する事項がある場合のみ「グループ財務ハンドブック」の関連箇所を参照し、該当する注意事項を「財務リマインド」に記載してください；キーワードのみで具体的な財務事項がない場合はReferenceを読み込まず、「なし」としてください。

この規則により、Agentは必要最小限の情報だけを読み込むことで、トークンの無駄を省きながら、会議の要約に財務上の重要な視点を追加することができます。これは、より賢く、より効率的なスキル利用への一歩となります。

Made with GAMMA

Agent Skillsの活用：Reference機能の実行テスト

「Reference」機能を組み込んだAgent Skillが正しく動作するか確認するため、財務関連の議論が含まれる会議のテキストを使ってテストを実行します。

Claude Codeに以下のプロンプトと会議内容を入力します。

以下の会議内容を要約してください。

会議のテキスト：
「田中：来月、新しい開発環境を構築するために、サーバーとPCを購入したいと思います。

佐藤：購入金額はどのくらいになりそうですか。

田中：サーバーが70万円、PCが40万円で、合計110万円程度を予定しています。

佐藤：100万円を超えるので、通常の購買承認だけではなく、管理部門と経営層の承認も必要か確認しましょう。

鈴木：見積書は複数社から取得していますか。

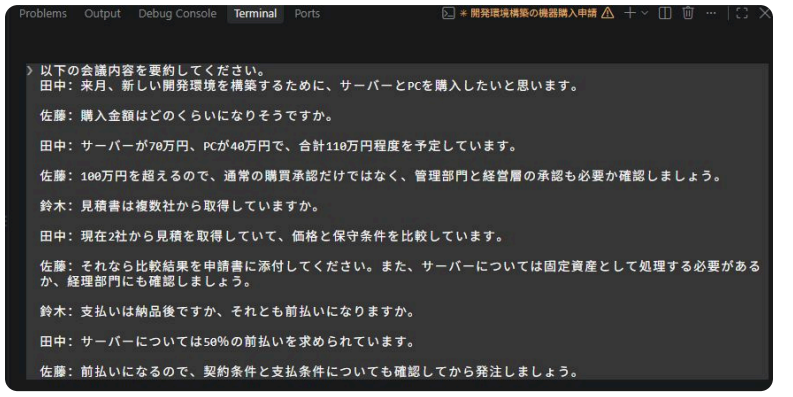
田中：現在2社から見積を取得していて、価格と保守条件を比較しています。

佐藤：それなら比較結果を申請書に添付してください。また、サーバーについては固定資産として処理する必要があるか、経理部門にも確認しましょう。

鈴木：支払いは納品後ですか、それとも前払いになりますか。

田中：サーバーについては50%の前払いを求められています。

佐藤：前払いになるので、契約条件と支払条件についても確認してから発注しましょう。
」



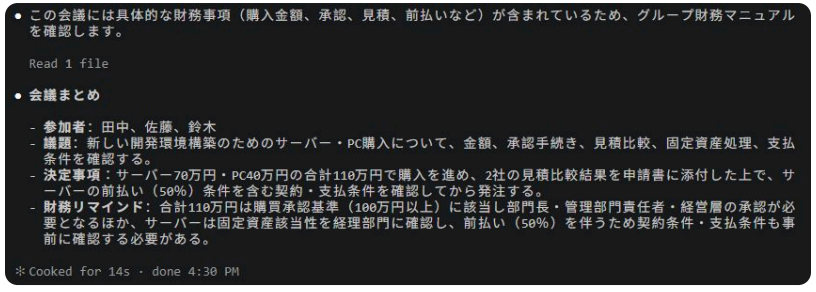
この会議では、新しい開発環境のためのサーバーとPC購入に関する議論が行われています。特に、100万円を超える費用や前払いといった財務関連の具体的な判断事項が含まれているため、Agent Skillが「グループ財務ハンドブック」を参照するはずです。

Made with GAMMA

Agent Skillsの活用：Reference機能の実行結果

私たちが定義したSKILL.mdと財務リマインド規則に従い、Claude Codeは以下のような会議要約を生成しました。特に、財務関連の議論があった場合に「グループ財務ハンドブック」から関連情報を抽出し、適切な「財務リマインド」を提供しています。

出力：
- 参加者: 田中、佐藤、鈴木
- 議題: 新しい開発環境のためのサーバーとPC購入について、金額、承認プロセス、見積、支払い条件を決定する。
- 決定事項: サーバー70万円、PC40万円の合計110万円の購入を予定しており、100万円超のため管理部門と経営層の承認が必要か確認する。現在2社から見積取得済みで比較結果を申請書に添付する。サーバーの固定資産処理の要否を経理部門に確認する。サーバーは50%前払いのため、契約条件と支払い条件を確認してから発注する。
- 財務リマインド: 購入金額が100万円を超えるため、部門長・管理部門責任者・経営層の承認が必要です。また、サーバーが固定資産に該当する可能性があり、50%の前払いについても支払条件の確認が必要です。



このように、「財務リマインド」が「なし」ではなく、具体的な注意点が記述されました。これは、Agentが会議内容を適切に判断し、必要なタイミングでReferenceドキュメント（グループ財務ハンドブック）を参照した結果です。これにより、ユーザーは会議の要約から重要な財務上の注意点を一度に把握でき、より迅速かつ正確な意思決定が可能になります。

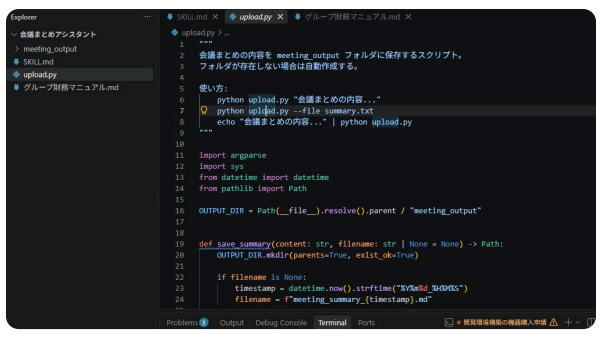
Made with GAMMA

Agent Skill の高度な使い方（Script編）

Made with GAMMA

「Script」の実装：upload.pyの追加

「Script」機能の活用を始めるために、まずはupload.pyというPythonスクリプトファイルを作成します。upload.py の機能は、生成した会議要約を指定したサーバーにアップロードすることです。



```
# upload.py
"""
会議まとめの内容を meeting_output フォルダに保存するスクリプト。
フォルダが存在しない場合は自動作成する。

使い方:
    python upload.py "会議まとめの内容..."
    python upload.py --file summary.txt
    echo "会議まとめの内容..." | python upload.py
"""

import argparse
import sys
from datetime import datetime
from pathlib import Path

OUTPUT_DIR = Path(__file__).resolve().parent / "meeting_output"


def save_summary(content: str, filename: str | None = None) -> Path:
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

    if filename is None:
        timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
        filename = f"meeting_summary_{timestamp}.md"

    output_path = OUTPUT_DIR / filename
    output_path.write_text(content, encoding="utf-8")
    return output_path


def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="会議まとめを meeting_output フォルダに保存する")
    parser.add_argument("content", nargs="?", help="保存する会議まとめの内容")
    parser.add_argument("--file", help="会議まとめの内容が書かれたファイルのパス")
    parser.add_argument("--name", help="保存するファイル名（省略時はタイムスタンプ付きで自動生成）")
    return parser.parse_args()


def main() -> None:
    args = parse_args()

    if args.file:
        content = Path(args.file).read_text(encoding="utf-8")
    elif args.content:
        content = args.content
    elif not sys.stdin.isatty():
        content = sys.stdin.read()
    else:
        print("エラー: 保存する内容を指定してください（引数 / --file / 標準入力）。", file=sys.stderr)
        sys.exit(1)

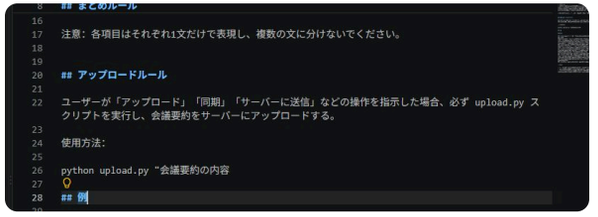
    output_path = save_summary(content, args.name)
    print(f"保存しました: {output_path}")


if __name__ == "__main__":
    main()
```

Made with GAMMA

「SKILL.md」にアップロード規則を追加

導入したupload.pyスクリプトを活用するため、次にSKILL.mdにアップロード規則を追加します。



この規則により、ユーザーが「アップロード」「同期」「サーバーに送信」といったキーワードを会議内容や指示の中で言及した場合、Agentは自動的に生成された会議要約をサーバーにアップロードするupload.pyスクリプトを実行します。

```
## アップロードルール

ユーザーが「アップロード」「同期」「サーバーに送信」などの操作を指示した場合、必ず upload.py スクリプトを実行し、会議要約をサーバーにアップロードする。

使用方法：

python upload.py "会議要約の内容
```

これにより、会議要約の保存・共有プロセスが自動化され、作業効率が大幅に向上します。

Made with GAMMA

Upload.pyテスト実施

以下の会議内容を要約し、サーバーにアップロードしてください。
田中：現在進めている新機能の開発状況について共有します。画面側の実装はほぼ完了しています。

佐藤：バックエンド側のAPI開発はどこまで進んでいますか。

田中：主要なAPIは実装済みで、現在はエラー処理とログ出力の部分を修正しています。

鈴木：結合テストはいつ頃開始できそうですか。

田中：来週月曜日から開始する予定です。テスト項目については、今週中に整理しておきます。

佐藤：テストで問題が見つかった場合は、優先度を付けて対応しましょう。

鈴木：了解しました。テスト結果は共有フォルダにまとめて、チーム全員が確認できるようにします。

佐藤：では、今週中にAPIの修正とテスト項目の整理を完了し、来週月曜日から結合テストを開始するということでお願いします。

Made with GAMMA

期待するClaude Codeの出力

参加者：田中、佐藤、鈴木 - 議題：新機能開発の進捗状況、API開発、結合テストの準備について確認する。 - 決定事項：今週中にAPIの修正とテスト項目の整理を完了し、来週月曜日から結合テストを開始する。 - 財務リマインド：なし

Made with GAMMA

Tips１：

今回使用した会議内容は、お金や財務に関係する内容ではなかったため、Claude Codeは「グループ財務マニュアル.md」の内容を読み込まず、結果にも財務リマインドに関する項目は含まれていませんでした。

このことから、Referenceは必要な場合にのみ読み込まれる「オンデマンド読み込み」であることが分かります。ユーザーがReferenceに関連する内容を扱っていない場合、Claude CodeはそのReferenceを読み込まないため、コンテキストのトークンを節約することができます。

Made with GAMMA

Tips2：

Scriptは、Referenceのように「内容を読み込んで理解する」のではなく、「実行する」ことが主な役割です。

そのため、1万行の複雑なロジックを持つScriptであっても、コード全体をモデルのコンテキストに読み込まずに実行できる場合、コンテキストトークンの消費を大幅に抑えることができます。

ただし、実行前にScriptの内容を確認する必要がある場合は、Claude CodeがScriptを読み込むこともあります。

つまり、**Scriptは「読むもの」ではなく「実行するもの」が基本ですが、必要に応じて読み込まれる場合もあります。**

Made with GAMMA

まとめ

ReferenceとScriptは、どちらもAgent Skillの高度な使い方ですが、モデルのコンテキストへの影響は大きく異なります。

Referenceは「読む」ものであり、内容がコンテキストに読み込まれるため、トークンを消費します。

一方、Scriptは「実行する」ことが主な役割であり、実行時に必ずしもコード全体をコンテキストに読み込む必要はありません。そのため、コード自体によるコンテキストトークンの消費を抑えることができます。

ただし、実行前にScriptの内容を確認する必要がある場合は、Claude CodeがScriptを読み込むこともあります。

Made with GAMMA



Made with GAMMA

Agent Skills VS MCP

Agent Skillsについて説明してきましたが、ここまで見てみると、Agent SkillsとMCPは少し似ていることに気づきます。

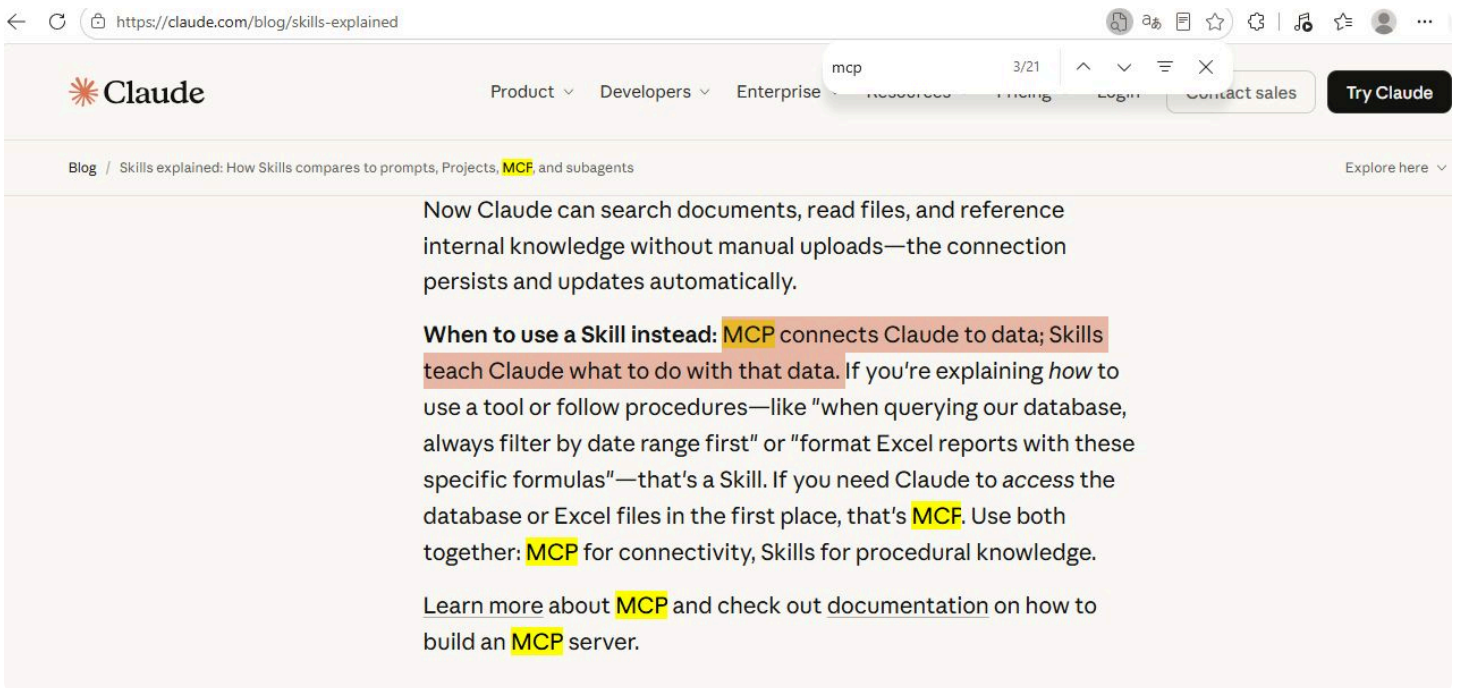
どちらも本質的には、**モデルが外部の世界とつながり、外部の情報やツールを利用できるようにする仕組み**です。

では、機能が重複しているように見える中で、**私たちはどちらを使えばよいのでしょうか？**

この点について、Anthropic社が詳しく解説した記事を公開しています。

Made with GAMMA

claude.com/blog/skills-explained



Made with GAMMA



Made with GAMMA

Agent SkillsとMCP：それぞれの本質と適した利用シーン

MCPの本質は「プログラムを実行する仕組み」であるのに対し、Agent Skillsの本質は「ドキュメント」です。この本質的な違いによって、それぞれ適した利用シーンも異なります。

Agent Skillsは、軽量なScriptの実行や、比較的シンプルな処理を行う用途に適しています。一方で、**安全性や安定性の面では、Agent SkillsよりもMCPのほうが適しています。** そのため、用途や処理内容に応じて、適切なツールを選択することが重要です。

Agent Skills：ドキュメントとしての柔軟性

Agent Skillsの本質は「ドキュメント」です。`SKILL.md`という形式で、モデルに何をすべきかを記述し、参照させることで、軽量なスクリプトの実行や比較的シンプルなデータ処理を効率的に行います。

- 軽量なスクリプト実行
- シンプルな情報抽出・加工
- 迅速なスキル定義と変更

MCP：プログラム実行による堅牢性

MCPの本質は「プログラムを実行する仕組み」です。外部ツールやAPIとの連携において、より高度な制御、安全性、安定性が求められる複雑なタスクに適しています。堅牢なシステム開発や、セキュリティが重要な業務での活用が期待されます。

- 安全性と安定性の確保
- 複雑な多段階タスク
- 外部システムとの堅牢な連携

Made with GAMMA

Agent Skillsの主な入手先

Agent Skillsは様々なプラットフォームから入手・発見することができ、用途や目的に応じて最適なチャネルを選ぶことが重要です。

チャネル	特徴	推薦度
Anthropic 公式	公式のSkills、ドキュメント、ベストプラクティス	⭐⭐⭐⭐⭐
GitHub	最大のコミュニティSkillsソース	⭐⭐⭐⭐⭐
skills.sh	Claude Code / Codex / Cursorなどエコシステムを横断	⭐⭐⭐⭐⭐
Claude Code Marketplace	プラグイン/スキルを直接インストール可能	⭐⭐⭐⭐
企業公式 GitHub	HashiCorpなど企業の専門スキル	⭐⭐⭐⭐
コミュニティ Marketplace / Directory	大量のSkillsを検索・発見しやすい	⭐⭐⭐

Made with GAMMA

Anthropic公式Agent Skillsのダウンロードと利用方法

Anthropicの公式Agent Skillsを利用するための基本的なコマンドと手順を以下に示します。



プラグインの表示と管理

`/plugin`

インストールされているすべてのプラグインを表示し、管理するための基本コマンドです。



Anthropic公式マーケットプレイスの追加

`/plugin marketplace add anthropics/skills`

Anthropic公式マーケットプレイスを追加して、利用可能なスキルを探索します。



必要なプラグイン/スキルのインストール

`/plugin install <plugin>@<marketplace>`

特定のAgent Skillをインストールするには、このコマンドを使用します。例：`/plugin install claude-api@anthropic-agent-skills`



プラグインのアンインストール

`/plugin uninstall <plugin>@<marketplace>`

不要になったプラグインは、上記のコマンドを使用して簡単に削除できます。

Made with GAMMA

Agent Skillsの活用：Claude Codeでの実行

Agent Skillsの一つである`claude-api`スキルが実際にどのように利用されるかを検証するため、Claude Codeに以下の質問を投げかけます。これにより、スキルのインストール状況、Python SDKの使用方法、そしてスキルが実際にロードされたかを確認します。

> 現在の環境で `claude-api` スキルがインストールされており、利用可能であるか確認してください。
> もしインストールが成功している場合、続けて質問します。
> PythonでClaude APIを呼び出す方法として、現在推奨されているAnthropic Python SDKのインストール方法を教えてください。
> そして、最もシンプルなAPI呼び出しのサンプルコードを書いてください。
> この質問に回答する際、`claude-api` スキルをロードしましたか？
> もしロードしたなら、そのスキルの名称を教えてください。

Made with GAMMA